



Computer Organization & Assembly Language

Course: Computer Organization & Assembly Language (Lab) - CS-318

1. Introduction to Arithmetic Instructions

The x86 architecture provides fundamental instructions for performing arithmetic operations: **ADD** (addition), **SUB** (subtraction), **MUL** (multiplication), and **DIV** (division).

These operations are essential for any computation and often utilize general-purpose registers like **AX**, **BX**, **CX**, and **DX** for both operands and results. This lab focuses on 16-bit operations.

2. ADD Instruction (Addition)

Syntax

ADD destination, source

Explanation

The **ADD** instruction computes the sum of the destination operand and the source operand. The result of the addition is then stored back into the **destination** register.

Destination ← Destination + Source

Example: Addition of Two 16-bit Numbers

Objective: Add the 16-bit numbers 2500h and 1500h and store the result in AX.

Code

```
ORG 100h
MOV AX, 2500h ; Load the first 16-bit number into AX
MOV BX, 1500h ; Load the second 16-bit number into BX
ADD AX, BX ; AX = AX + BX
MOV AH, 4Ch ; Function to terminate program
INT 21h ; Call DOS interrupt
```

Step-by-step Execution

- **MOV AX, 2500h:** The accumulator register AX is initialized to 2500h.
- **MOV BX, 1500h:** The base register BX is initialized to 1500h.
- **ADD AX, BX:** The value of BX (1500h) is added to the value of AX (2500h).
- $AX \leftarrow 2500h + 1500h = 3A00h$

Result

AX=3A00h (Decimal equivalent: 14848)

3. SUB Instruction (Subtraction)

Syntax

```
SUB destination, source
```

Explanation

The **SUB** instruction computes the difference between the destination operand and the source operand (destination - source). The result of the subtraction is then stored back into the **destination** register.

```
Destination ← Destination - Source
```

Example: Subtraction of Two 16-bit Numbers

Objective: Subtract the 16-bit number 2500h from 4500h and store the result in AX.

Code

ORG 100h

MOV AX, 4500h ; Load the first 16-bit number (Minuend) into AX

MOV BX, 2500h ; Load the second 16-bit number (Subtrahend) into BX

SUB AX, BX ; AX = AX - BX

MOV AH, 4Ch

INT 21h

Step-by-step Execution

- **MOV AX, 4500h**: AX is initialized to 4500h.
- **MOV BX, 2500h**: BX is initialized to 2500h.
- **SUB AX, BX**: The value of BX (2500h) is subtracted from the value of AX (4500h).
- $AX \leftarrow 4500h - 2500h = 2000h$

Result

AX=2000h (Decimal equivalent: 8192)

4. MUL Instruction (Unsigned Multiplication)

Syntax

MUL source

Explanation

The **MUL** instruction performs unsigned multiplication. For 16-bit multiplication, the implicit operation is **AX×Source**.

Operand Size	Multiplicand (Implicit)	Multiplier (Source)	Product (Implicit)
16-bit	AX	Source (e.g., BX)	DX:AX (32-bit result)

The 32-bit product is stored across two 16-bit registers: the high word (most significant 16 bits) in **DX** and the low word (least significant 16 bits) in **AX**.

Example: Multiplication of Two 16-bit Numbers

Objective: Multiply 0006h by 0005h.

Code

```
ORG 100h
MOV AX, 0006h ; Multiplicand (implicitly AX)
MOV BX, 0005h ; Multiplier (source operand)
MUL BX ; DX:AX = AX * BX
MOV AH, 4Ch
INT 21h
```

Step-by-step Execution

- **MOV AX, 0006h:** AX is initialized to 6.
- **MOV BX, 0005h:** BX is initialized to 5.
- **MUL BX:** Multiplies AX by BX.
- **DX:AX** ← 6 × 5 = 30 decimal or 1Eh

Since 30 is small and fits within 16 bits, the result in hexadecimal is 001Eh. The high word is zero.

Result

DX=0000h (High word)

AX=001Eh (Low word - Decimal 30)

5. DIV Instruction (Unsigned Division)

Syntax

DIV source

Explanation

The **DIV** instruction performs unsigned division. For 16-bit division, the dividend is a 32-bit number implicitly stored in **DX:AX**, and the divisor is the **Source** operand.

Operand Size	Dividend (Implicit)	Divisor (Source)	Quotient (Implicit)	Remainder (Implicit)
16-bit	DX:AX (32-bit)	Source (e.g., BX)	AX (16-bit)	DX (16-bit)

The **quotient** is stored in **AX**, and the **remainder** is stored in **DX**.

Example: Division of Two 16-bit Numbers

Objective: Divide 48 (decimal, 0030h) by 6 (decimal, 0006h).

Code

```
ORG 100h
MOV AX, 0030h ; Lower 16 bits of dividend (48 decimal)
MOV DX, 0000h ; Higher 16 bits of dividend (must be 0 for a 16-bit dividend)
MOV BX, 0006h ; Divisor (6 decimal)
DIV BX ; DX:AX / BX → AX=Quotient, DX=Remainder
MOV AH, 4Ch
INT 21h
```

Step-by-step Execution

- **MOV DX:AX = 0000:0030h:** The 32-bit dividend is set to 00000030h (48 decimal).
- **MOV BX, 0006h:** The divisor BX is set to 0006h (6 decimal).
- **DIV BX:** Division operation 48/6.
- **Quotient (AX) ← 8 (0008h)**
- **Remainder (DX) ← 0 (0000h)**

Result

AX=0008h (Quotient - Decimal 8)

DX=0000h (Remainder - Decimal 0)

Operation	Instruction	Example	Result Register(s)	Comment
Addition	ADD	ADD AX, BX	AX	AX = AX + BX
Subtraction	SUB	SUB AX, BX	AX	AX = AX - BX
Multiplication	MUL	MUL BX	DX:AX	DX:AX = AX × BX
Division	DIV	DIV BX	AX (Q), DX (R)	AX = Quotient, DX = Remainder

Lab Exercise

Addition (ADD)

- **Scenario:** The value 1000h is added to 2000h. The result is stored in AX.
Predicted Result: AX=?
- **Scenario:** The value 0100h is added to 3F00h. The result is stored in AX.
Predicted Result: AX=?

Subtraction (SUB)

- **Scenario:** 0050h is subtracted from 00A0h. The result is stored in BX. **Predicted Result:** BX=?
- **Scenario:** 1F00h is subtracted from 2000h. The result is stored in AX. **Predicted Result:** AX=?
- **Scenario:** 0001h is subtracted from 0000h. The result is stored in AX (This is -1 in two's complement). **Predicted Result:** AX=?

Multiplication (MUL)

- **Scenario:** The number 10h is multiplied by 03h. The 32-bit product is stored in DX:AX. **Predicted Result:** DX=?, AX=?
- **Scenario:** The number 100 decimal (64h) is multiplied by 4. The 32-bit product is stored in DX:AX. **Predicted Result:** DX=?, AX=?

Division (DIV)

- **Scenario:** 20h is divided by 04h. The quotient is stored in AX, and the remainder in DX. (The dividend is 0000:0020h). **Predicted Result:** AX=? (Quotient), DX=? (Remainder)
 - **Scenario:** 1Ah (26 decimal) is divided by 05h (5 decimal). The quotient is stored in AX, and the remainder in DX. **Predicted Result:** AX=? (Quotient), DX=? (Remainder)
 - **Scenario:** 1000h is divided by 10h. The quotient is stored in AX, and the remainder in DX. **Predicted Result:** AX=? (Quotient), DX=? (Remainder)
-